

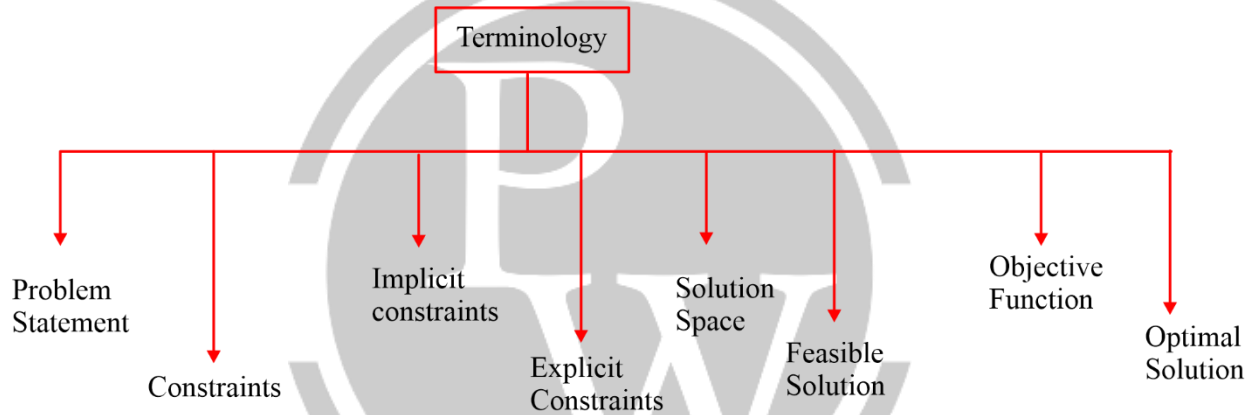
4

DYNAMIC PROGRAMMING

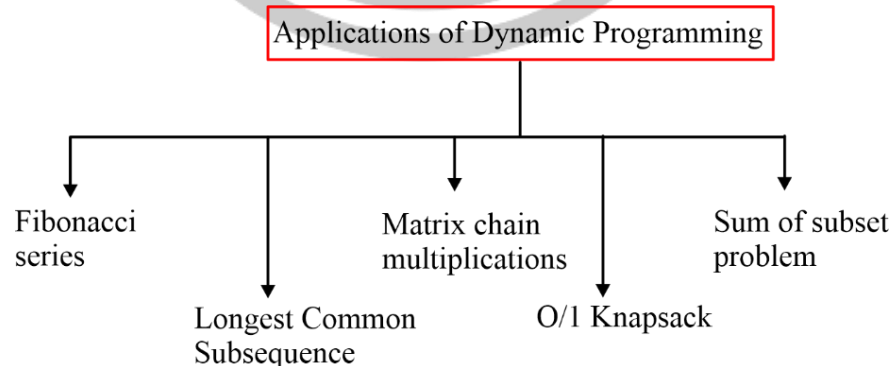
4.1 Dynamic Programming

In dynamic programming for optimal solution always computes distinct function calls.

4.2 Terminology



4.3 Application of Dynamic Programming

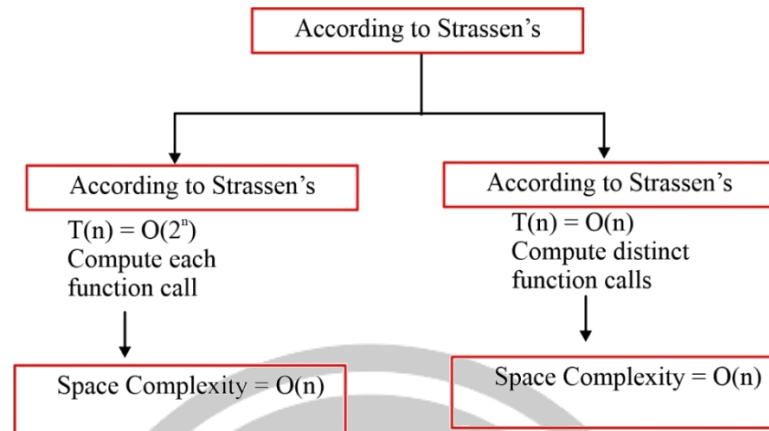


4.4 Fibonacci Series

- Time complexity $T(n) = O(n \log n)$
- Computes distinct function calls.

4.5 Job Sequence with Deadline

- Single CPU only
- Arrival time of each job is same
- No pre-emption



4.6 Longest Common Sub sequence (LCS)

- For common subsequence always consider two strings:
- $P = \langle ABCDB \rangle - Q = \langle BDCABA \rangle$
- Common subsequences for both 'p' are
- $S = \langle A \rangle$
- $S = \langle AB \rangle$
- $S = \langle CAB \rangle$
- $S = \langle BDAB \rangle$
- A common subsequence of longest length is known as longest common subsequence.
- For above problem longest common subsequence will be of length u.

4.6.1 Applications of LCS

1. Genomics
2. Software engineering applications
3. Plagiarism
4. Data gathering system of search engines

4.6.2 Algorithm for LCS

LCS (p,q)

{

1. For $i \leftarrow 0$ to $n-1$
 $L[i-1] = 0$
2. For $j \leftarrow 0$ to $m-1$
 $L[-1, j] = 0$

```

3. For i ← 0 to n-1
   For j ← 0 to m-1
     If (p[i] = q[j]) then
       L[i, j] = 1 + L(i-1, j-1);
     else
       L[i, j] = max{L[i, j-1], L[i-1, j]}
   }

```

- Time complexity of step 1 = $O(n)$
- Time complexity of step 2 = $O(m)$
- Time complexity of step 3 = $O(mn)$
- Total Time complexity = $O(n) + O(m) + O(mn)$
= $O(mn)$
- Space complexity = $O[(M+1).(n+1)]$
= $O(mn)$

4.7 Matrix Chain Multiplications

Two matrices 'A' and 'B' are compatible if and only number of column of first matrix must be equal to number of rows of second matrix.

4.7.1 Brute force method

Number of parenthesizing for a given chain is given by Catalan number: $\left[\frac{1}{n+1} {}^{2n}C_n \right]$

Time complexity = $O(n^n)$

Space complexity = $O(n)$

4.7.2 Algorithm For Matrix Chain Multiplication

The time complexity of multiply the given chain of n matrices $\langle A_1, A_2, A_3, \dots, A_n \rangle$ using dynamic programming (district function call) is $O(n^3)$

Space complexity = $O(n^2)$

